

> loading_talk()

Agentes de 0 a producción

// Con Strands Agents y Amazon Bedrock Agentcore.

00

// Clase

Pre-work

Fundamentos de LLMs, agentes y setup. Asincrónico, antes de la clase 1.

// 5 sessions, 4 etapas

Las 4 etapas del curso

01

Strands

Tools, Agent Loop, multi-provider.
Bedrock, Anthropic, OpenAI, Ollama.

02

MCP

Servidor MCP propio sobre Duffel.
Tools reusables entre agentes.

03

AgentCore

Runtime, Memory persistente,
Gateway corporativo. Todo en AWS.

04

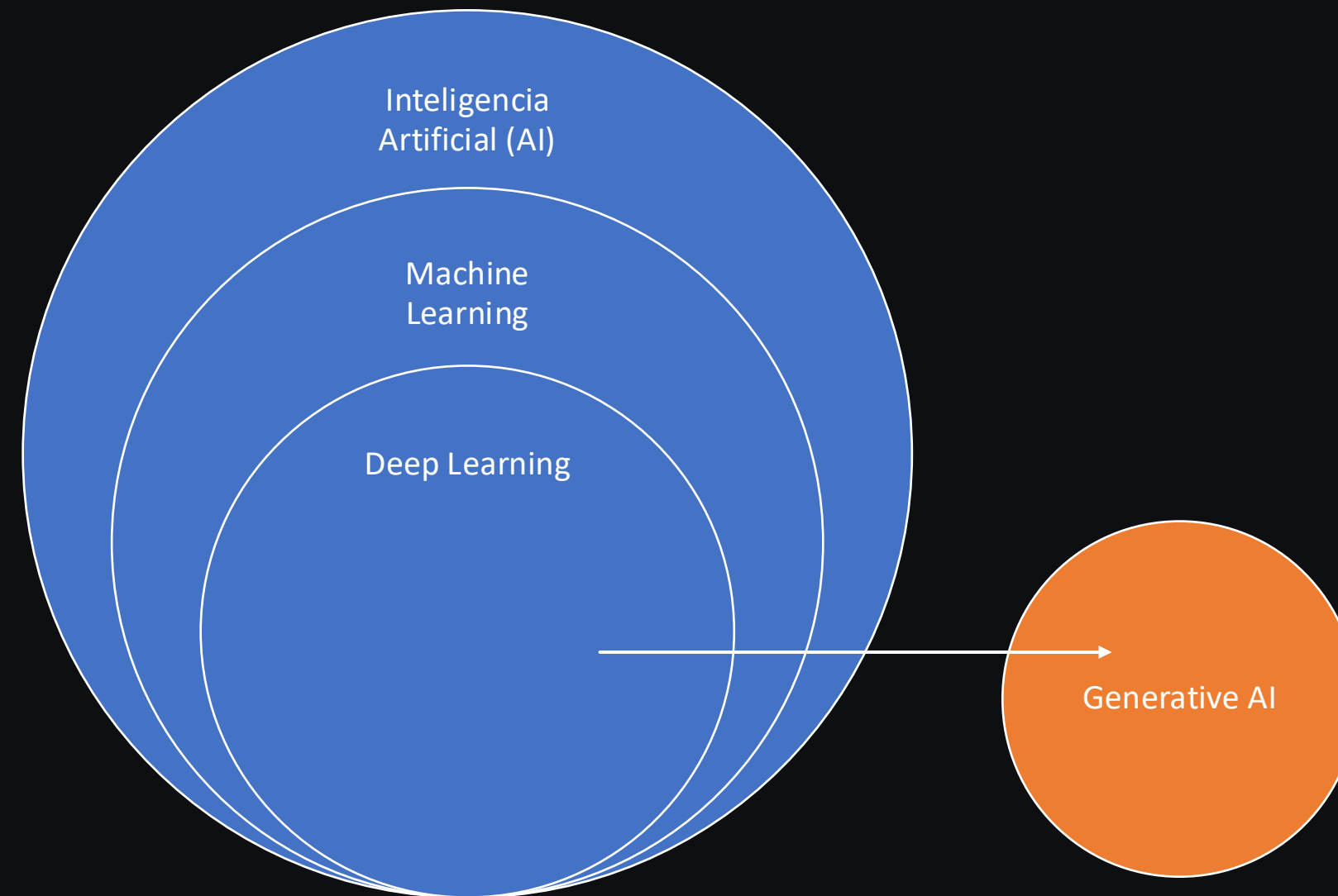
Producción

Policies, hooks, webhooks, deploy.
Listo para usuarios reales.

```
function whatIsAnLLM() {
```

De que hablamos cuando hablamos de **GenAI**?

```
}
```



```
function whatIsAnLLM() {
```

Que es GenAI?

```
}
```

> Pregunta: Que es Generative AI?

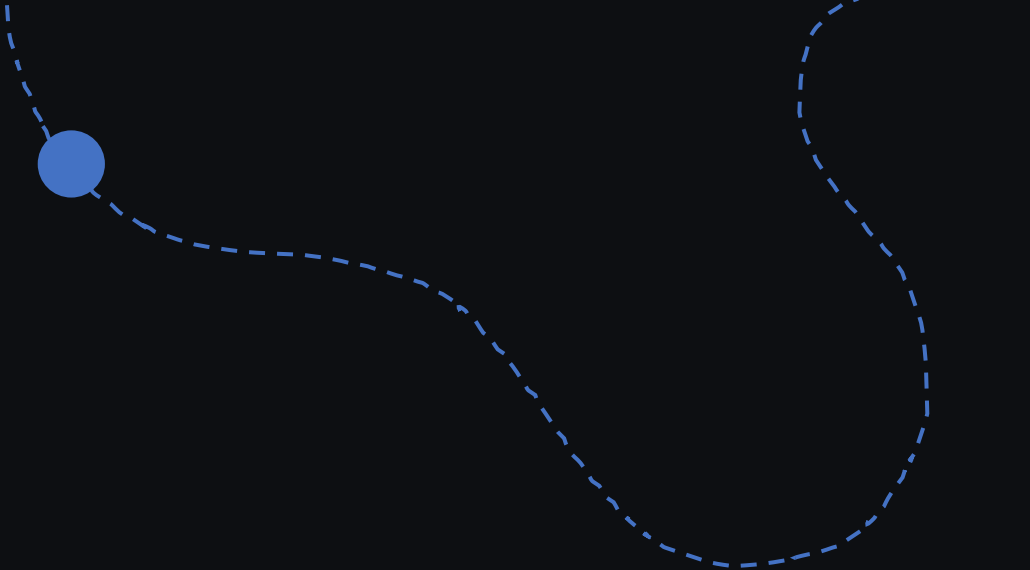
> Generative AI es una rama de la IA que se focaliza en crear nueva información. Es un subset de Machine Learning. El objetivo de la IA generative es crear nueva información que es similar a la información que fue utilizada para crear el modelo.

```
function whatIsAnLLM() {
```

Qué es un LLM?

```
}
```

Word Embeddings

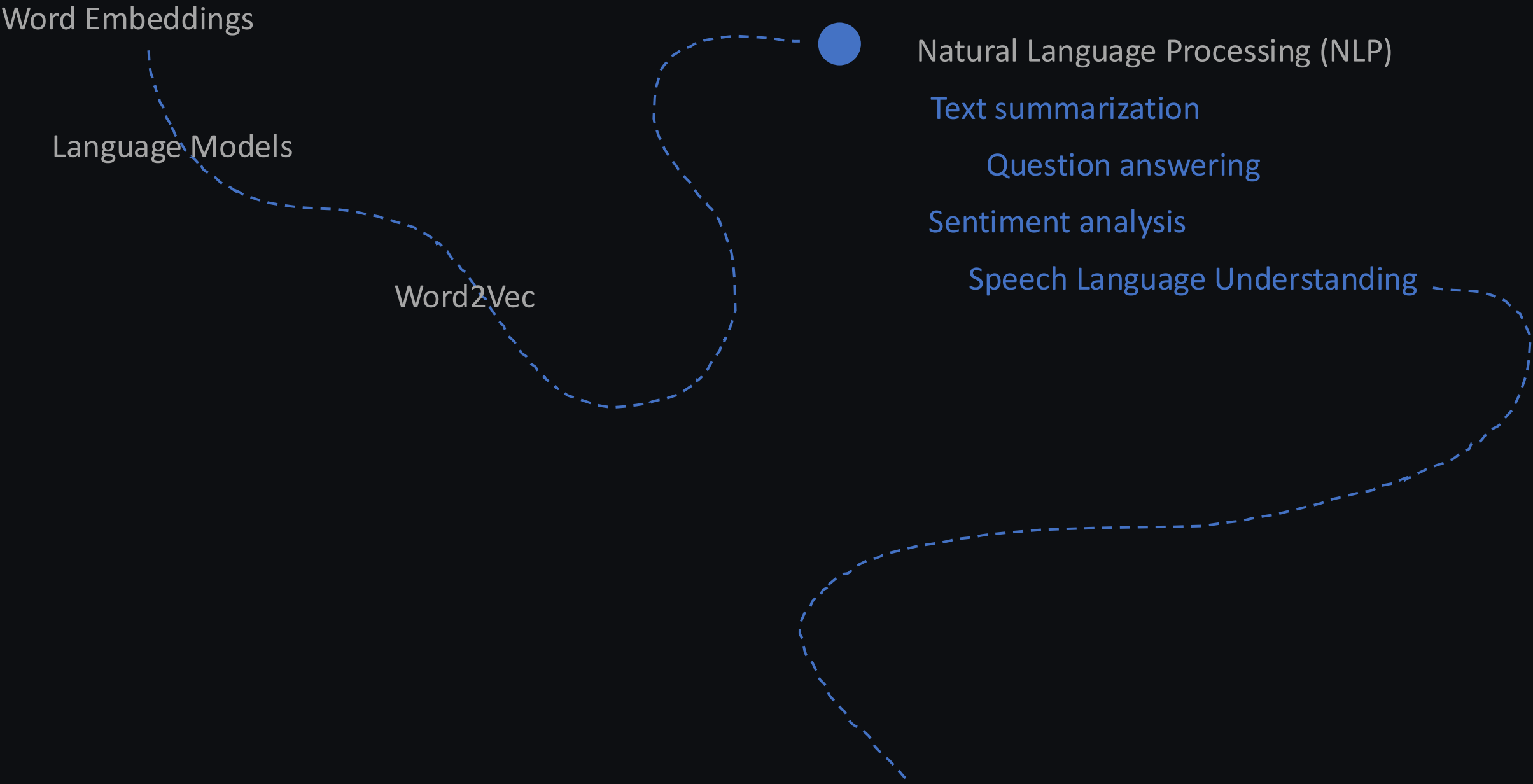


		traffic	vehicle
		car	
	drive	driver	bus
walk	person		
dog	animal	tea	
cat			

```
function whatIsAnLLM() {
```

Qué es un LLM?

```
}
```



```
function whatIsAnLLM() {
```

Que es un LLM.

```
}
```

Predicción de la siguiente palabra: word embedding.

Sentence:

?

q

?

ov

?

u

?

o

?

la

?

?

person.

```
function whatIsAnLLM() {
```

Que es un LLM.

```
}
```

Predicción de la siguiente palabra, con Memoria:
Recurrent Neural Networks (RNN).

Sentence:

? ? w ? ? ver the lazy _

~~person.~~

rabbit.

Transformers - Attention Is All You Need - 2017

El modelo de arquitectura de transformers

- Los modelos con “Atención” son capaces de entender la relación entre las palabras.
- Hacen uso eficiente de parallel computing

“Attention is all you need”

Vaswani, Ashish & Shazeer, Noam & Parmar, Niki & Uszkoreit, Jakob & Jones, Llion & Gomez, Aidan & Kaiser, Lukasz & Polosukhin, Illia. (2017)

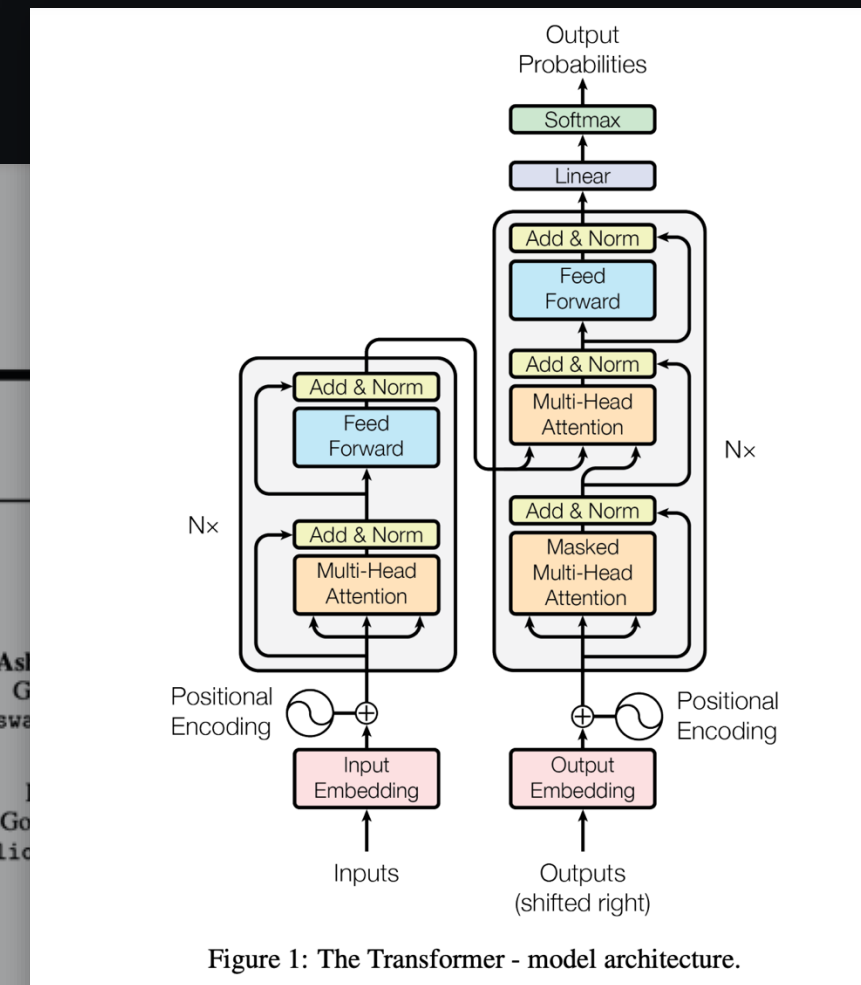


Figure 1: The Transformer - model architecture.

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.0 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature.

1 Introduction

Recurrent neural networks, long short-term memory [12] and gated recurrent [7] neural networks

```
function whatIsAnLLM() {
```

Que es un LLM.

```
}
```

Predicción de la siguiente palabra, con Memoria:

Transformers

Typing Tips

The quick brown fox jumps over the lazy **dog**. **Is an English-language pangram — a sentence that contains all the letters of the alphabet. The phrase is commonly used for touch-typing practice.**

Large Language Models (**LLM**)



```
function whatIsAnLLM() {
```

Que es un LLM.

```
}
```

// Un modelo entrenado para predecir el siguiente token dado un contexto. No razona como humano: estima distribuciones de probabilidad sobre vocabularios. Conceptos clave: tokens, context window, temperatura, costo por token. Un LLM solo no es un agente.

// 4 prerequisites

Setup **antes** de empezar

01

Python

Python 3.11+ con virtualenv. Sin esto no podemos usar Strands.

02

AWS

Cuenta AWS con Bedrock y DynamoDB. Region us-east-1. AWS CLI configurado.

03

Duffel

Cuenta sandbox de Duffel. API key gratis, sin tarjeta. Desde la clase 1.

04

Ollama

Opcional: Ollama instalado para la variante 100% local de la clase 1.

01

// CHAPTER

Strands Fundamentals

Agent Loop, tools con @tool, multi-provider. Construimos las tres tools del agente: search_flights, get_weather, save_search_log.

```
function whatIsAnAgent() {
```

LLM vs Agente.

```
}
```

¿Qué diferencia hay entre un LLM y un Agente?



LLM (Modelo)

- 📄 Predice texto en base a un prompt
- 🚫 No tiene memoria ni herramientas
- 💬 Solo responde con lenguaje
- 🤖 Ejemplo: "Clima en Buenos Aires" → *inventa*



Agente

- 🧠 Usa el LLM como cerebro
- 🔧 Tiene tools/APIs para actuar
- 📁 Planifica, razona y ejecuta pasos
- 🤖 Ejemplo: "Clima en Buenos Aires" → *consulta API de clima*

```
function whatIsAnAgent() {
```

LLM vs Agente.

```
}
```

// Un agente es un LLM con tools, memoria y un loop. Recibe un objetivo, decide acciones, observa resultados, y corrige hasta cumplirlo. El agente decide cuando llamar tools. Vos definis las tools.

// the agent loop

El Agent Loop



// El agente itera think-act-observe hasta cumplir el objetivo. Strands maneja el loop por nosotros.

```
function case() {
```

Corporate Travel Agent.

```
}
```

// Construimos un agente que ayuda a empleados a buscar y reservar vuelos. Busca en Duffel, valida con politicas corporativas, persiste reservas en DynamoDB. El caso evoluciona clase a clase: arrancamos con tools locales, terminamos deployando en AgentCore con observabilidad y webhooks.

```
function whatIsStrands() {
```

Que es Strands.

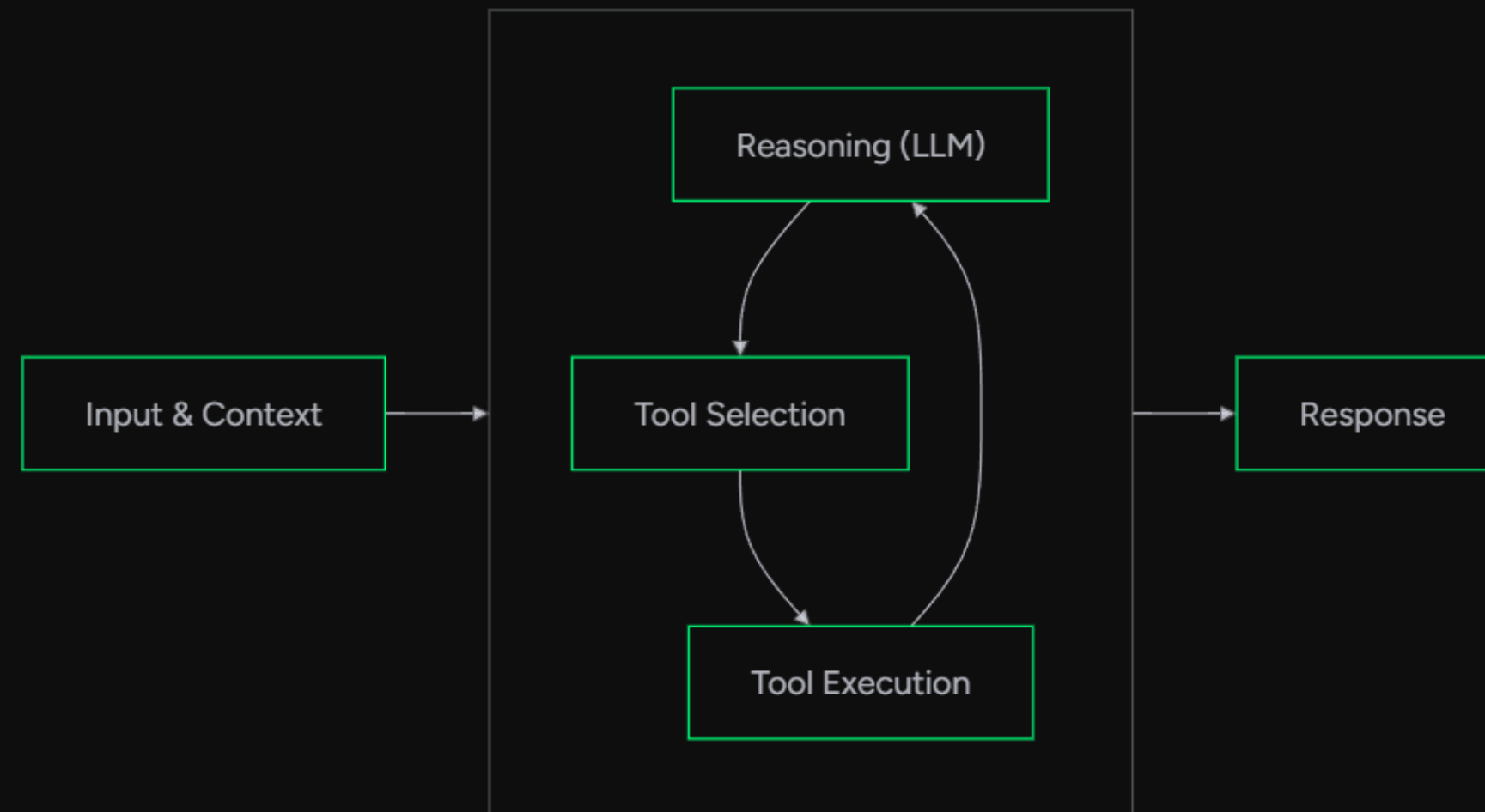
```
}
```



// Framework open-source de AWS para construir agentes con minimo boilerplate. Crear tools,. El loop think-act-observe lo maneja el framework. Multi provider de fabrica: Bedrock, Anthropic, OpenAI, Ollama, modelos locales.

// strands runtime

El Agent Loop en Strands



// Strands itera entre el modelo y las tools hasta que el modelo deja de pedir tool calls.

// 3 líneas para tu primer agente

Hello Agent

~/travel-agent/hello.py



```
from strands import Agent
```

```
agent = Agent()
```

```
# 3 líneas y ya tenemos un agente  
response = agent('Hola, quien sos?')  
print(response)
```

```
function whatIsATool() {
```

Tools = el primer superpoder.

```
}
```

// Una tool es una funcion Python decorada con @tool. El docstring es lo unico que el LLM ve para decidir cuando llamarla. Regla: si la accion requiere datos externos, side effects, o calculos deterministas, hacela tool. Si es solo razonamiento, dejala al LLM.

```
function whatIsATool() {
```

Tools en Strands.

```
}
```

// En Strands tenemos:

- Tools Nativas
- Tools de la comunidad
- Custom Tools

```
~/travel-agent/tools/flights.py
```

```
from strands import tool
```

```
import requests
```

```
@tool
```

```
def search_flights(origin: str, destination: str, date: str) -> dict:
```

```
    """Search flights for the given route and date.
```

```
    The LLM reads this docstring to decide WHEN to call this tool.
```

```
    Args: origin: IATA code (3 letters). destination: IATA code. date: YYYY-MM-DD."""
```

```
    response = requests.post("https://api.duffel.com/...", json={...})
```

```
    return response.json()
```

// el LLM solo lee el docstring

Bad vs Good

[- BAD]

```
def fn(x, y):
```

```
    """Search."""
```

El LLM no sabe que parametros pasar

No sabe cuando usarla

El agente la ignora o la usa mal

[+ GOOD]

```
def search_flights(origin, destination,  
date):
```

```
    """Search one-way flights via Duffel."""
```

Args con tipo y descripcion

El LLM sabe cuando llamarla

Retorna estructura previsible

// search via Duffel sandbox

Tool: **search_flights**

~/travel-agent/tools/flights.py



```
@tool
def search_flights(origin, destination, date):
    """Search one-way flights using Duffel."""
    headers = {'Authorization': f'Bearer {API_KEY}'}
    payload = {"slices": [{"origin": origin, ...}]}
    r = requests.post(API_URL, headers=headers, json=payload)
    offers = r.json()['data']['offers']
    return sorted(offers, key=lambda o: o['total_amount'])[:5]
```

```
// open-meteo, sin auth
```

Tool: `get_weather`

```
@tool
```

```
def get_weather(city: str, target_date: str) -> dict:
```

```
    """Forecast for a city on a specific date."""
```

```
    coords = geocode(city)
```

```
    r = requests.get(OPEN_METEO_URL, params={
```

```
        "latitude": coords["lat"], "longitude": coords["lon"],
```

```
        "daily": "temperature_2m_max,precipitation_sum"})
```

```
    return r.json()['daily']
```

// 4 providers de Strands

Multi-provider

PROVIDER	BEDROCK	OPENAI	OLLAMA	NOTES
Setup	AWS CLI	API key	local	// que necesita cada uno
Costo	3.75	15	Free	// USD por 1M output
Latencia	300ms	800ms	50ms	// p50 desde LATAM
Privacidad	AWS	Cloud	Local	// donde corre
Recomendado	Curso	Prod	Dev	// para este curso

// 100% local, sin API keys

Variante Ollama

```
~/travel-agent/class-1-  
ollama/agent.py
```



```
from strands import Agent  
from strands.models.ollama import OllamaModel  
  
model = OllamaModel(  
    host="http://localhost:11434", model_id="qwen3:8b")  
  
agent = Agent(model=model, tools=[search_flights, get_weather])  
agent('Vuelos a Miami el 15 de agosto')
```

02

// CHAPTER

MCP + Memoria

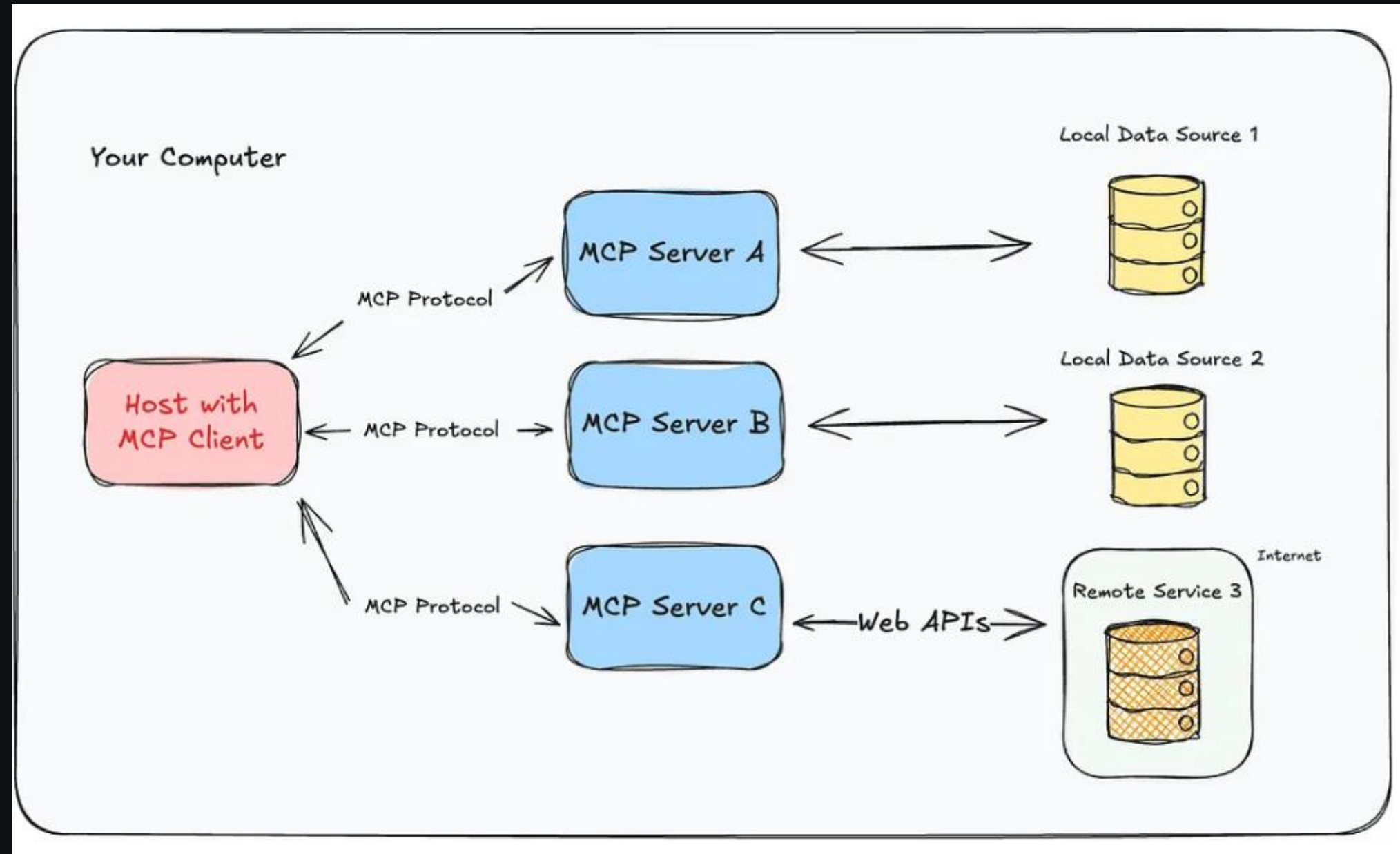
MCP estandariza como los agentes consumen tools. Mem0 le da memoria persistente. Construimos ambos y los integramos al agente.

```
function whyMCP() {
```

Que problema resuelve MCP.

```
}
```

```
// Antes de MCP, cada framework de agentes tenia su propio formato de tools. MCP es un
protocolo abierto que estandariza como cualquier cliente (agente, IDE, app) consume tools de
cualquier servidor. Resultado: tu MCP server lo usa Strands, Claude Desktop, Cursor, y
cualquier otro cliente compatible. Sin duplicar logica.
```



<https://modelcontextprotocol.io>

>

```
function mcpPrimitives() {
```

Las 3 primitivas de MCP.

```
}
```

// Tools: funciones que el agente ejecuta. Ya las conocemos. Resources: datos que el server expone como contexto, sin ejecutar nada. ejecutar nada. Prompts: templates reutilizables con variables. Ejemplo Resource: `@mcp.resource('airports://list')` expone la lista de la lista de aeropuertos IATA. El agente la lee sin llamar una API. Ejemplo Prompt: `@mcp.prompt('search-flight')` genera un template con template con origin, destination y date como variables. Nosotros nos enfocamos en Tools.

```
// cuando elegir cada uno
```

Tool local vs MCP tool

```
[ LOCAL ]
```

Funcion @tool en el codigo.

```
Vive dentro del proceso del agente  
Cero red entre agente y tool  
No reusable por otros clientes  
Ideal para logica simple, especifica
```

```
[ MCP ]
```

Servidor MCP separado.

```
Corre como proceso, contenedor o servicio  
Stdio o HTTP entre cliente y server  
Reusable por cualquier cliente MCP  
Ideal para tools que valen la pena compartir
```

// strands + MCP

Strands + MCP architecture



// El agente se conecta al MCP server via HTTP. El server traduce las tool calls a requests contra la API de Duffel.

```
~/travel-agent/mcp_server/duffel_server.py
```

```
from mcp.server.fastmcp import FastMCP
import requests

mcp = FastMCP("duffel-travel")

@mcp.tool()
def search_flights(origin: str, destination: str, date: str) -> dict:
    """Search one-way flights via Duffel sandbox."""
    return _call_duffel(origin, destination, date)
```

```
// el agente consume el MCP server
```

Strands + MCP

```
~/travel-agent/clase-2-mcp/agent.py
```



```
from mcp.client.streamable_http import streamable_http_client
from strands.tools.mcp import MCPClient
```

```
mcp_client = MCPClient(lambda: streamable_http_client("http://localhost:8002/mcp"))
```

```
with mcp_client:
    agent = Agent(tools=client.list_tools_sync())
```

```
function reuseEverywhere() {
```

Tu server, cualquier cliente.

```
}
```

// El MCP server que acabamos de construir no es solo para Strands. Lo puedes conectar a Claude Desktop, a Cursor, a cualquier cliente MCP. Escribis una vez, lo usas desde cualquier herramienta. Esto es lo que diferencia MCP de tools locales: la reusabilidad. Tu logica de negocio vive en un solo lugar.

Memoria con Mem0.

```
function noMemory() {
```

El agente no recuerda.

```
}
```

// Pedile un vuelo. Cerrá la sesión. Volvé a abrirla. Pedile otro vuelo. No sabe quién sos, no sabe que preferís pasillo, no sabe que sos sabe que sos vegetariano. Arranca de cero cada vez. Para un demo está bien. Para un producto que alguien use todos los días, no funciona. días, no funciona. Necesitamos memoria persistente.

```
function whatIsMem0() {
```

Que es Mem0.

```
}
```

// Mem0 es una librería open source de memoria para agentes. Le das un user_id, guardas hechos sobre ese usuario, y los recuperas cuando vuelve. Persiste entre sesiones. Lo usamos hoy porque es simple y open source. En clase 3 lo migramos a AgentCore Memory, el servicio gestionado de AWS.

// segunda sesión del mismo usuario

Sin memoria vs con memoria

[- SIN]

Sesion 2: arranca de cero.

- Pregunta otra vez aerolinea preferida
- No sabe que sos vegetariano
- Sugiere asiento del medio
- Tu paciencia se acaba

[+ CON]

Sesion 2: ya sabe quien sos.

- Aerolinea preferida cargada
- Restricciones alimentarias en contexto
- Asiento de pasillo por defecto
- El agente parece util de verdad

```
function memoryRoadmap() {
```

Mem0 hoy, AgentCore despues.

```
}
```

// Hoy usamos Mem0 porque es open source, simple, y no necesita infraestructura de AWS. Nos deja experimentar con memoria sin configurar nada en la nube. En clase 3 migramos a AgentCore Memory: el servicio gestionado de AWS. Misma idea, pero con IAM, multi-tenant nativo, y sin mantener infraestructura propia.

```
// el agente consume el MCP server
```

Strands + MCP

```
~/travel-agent/class-2-  
mcp/agent.py
```



```
from mcp import StdioServerParameters, stdio_client  
from strands.tools.mcp import MCPClient  
  
params = StdioServerParameters(  
    command="python", args=["mcp_server/duffel_server.py"]  
)  
  
with MCPClient(lambda: stdio_client(params)) as client:  
    agent = Agent(tools=client.list_tools_sync())
```

03

// CHAPTER

Strands Avanzado + AgentCore

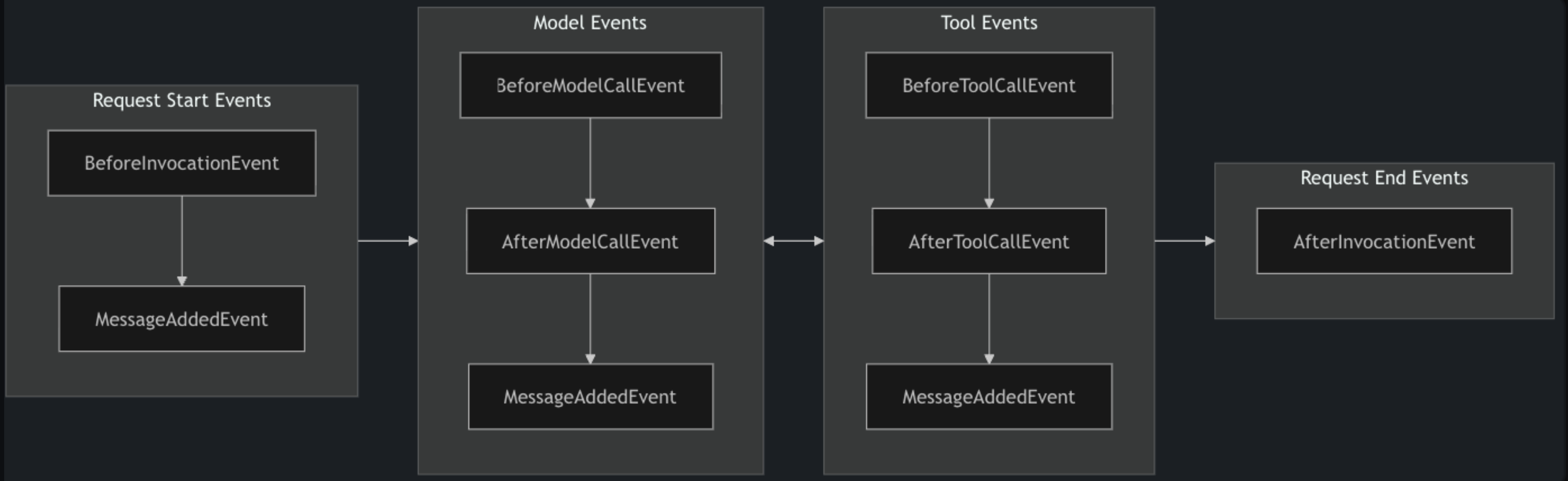
Hooks, Steering, "Multi agente", Introducción a Agentcore

```
function whatAreHooks() {
```

Hooks: interceptar el agent loop.

```
}
```

// Los hooks son callbacks que Strands ejecuta en 4 momentos del agent loop: antes y despues de cada invocacion, y antes y despues de antes y despues de cada tool call. Sin tocar la logica del agente. Casos de uso: loggear cada tool call (observabilidad), bloquear tools bloquear tools peligrosas (enforcement), medir latencia, auditar, inyectar contexto.



```
~/travel-agent/hooks/observability.py
```

```
from strands.hooks import HookProvider, HookRegistry
from strands.hooks.events import BeforeToolInvocationEvent, AfterToolInvocationEvent

class ObservabilityHook(HookProvider):
    def register_hooks(self, registry: HookRegistry):
        registry.add_callback(BeforeToolInvocationEvent, self.before)
        registry.add_callback(AfterToolInvocationEvent, self.after)

    def before(self, event):
        log.info("tool.start", tool=event.tool_use.name)
```

```
~/travel-agent/hooks/enforcement.py
```

```
class EnforcementHook(HookProvider):  
    def register_hooks(self, registry):  
        registry.add_callback(BeforeToolInvocationEvent, self.gate)  
  
    def gate(self, event):  
        if event.tool_use.name == "search_flights":  
            dest = event.tool_use.input.get("destination")  
            if dest in RESTRICTED_DESTINATIONS:  
                event.skip_invocation(  
                    reason=f"Destino {dest} restringido por politica")
```

```
function whatIsSteering() {
```

Steering: guiar sin hardcodear.

```
}
```

// Un plugin de Strands que inyecta instrucciones intermedias entre turnos del agent loop. Guia al agente sin modificar el system prompt original. Ideal para forzar patrones de comportamiento. Ejemplo: despues de cada tool call, inyectar 'siempre confirma con el usuario antes de reservar'. El agente lo cumple sin que vos toques la logica.

```
function steeringMoment() {
```

Antes de llamar a una Tool

```
}
```



```
function steeringMoment() {
```

Después de que el modelo responda

```
}
```



```
function multiAgentPatterns() {
```

Multi-agente: 4 patrones.

```
}
```

// Agents as Tools: un agente usa a otro como tool.

Swarm: grupo de agentes que se delegan tareas.

Graph: flujo dirigido entre agentes.

A2A: protocolo abierto de Google para agentes remotos.

```
function multiAgentPatterns() {
```

Agent as Tool.

```
}
```

Cuando usarlo:

- Tienes un agente orquestador que necesita habilidades especializadas.
- Los sub-agentes viven en el mismo proceso Python (mismo deploy, mismo equipo).
- Quieres control explícito: el orquestador decide cuándo y cómo delegar.

*Señales de que es el patron correcto:

- "El travel agent necesita consultar clima, pero no quiero que sepa de meteorología."
- "Quiero separar responsabilidades sin complejidad de red."

Señales de que NO es el patron correcto:

- Los sub-agentes los mantiene otro equipo o corren en otro servicio → usar A2A.
- No sabes de antemano qué agente debe tomar cada parte → usar Swarm.

```
function multiAgentPatterns() {
```

Swarm.

```
}
```

Cuando usarlo:

- La tarea se puede descomponer pero no sabes de antemano el orden óptimo.
- Los agentes tienen distintas especialidades y pueden pasarse el trabajo entre ellos.
- Aceptas que el framework decide los handoffs.

Señales de que es el patron correcto:

- "Necesito vuelos, clima y un resumen, pero no me importa el orden exacto."
- Prototipado rapido: querés ver si los agentes se coordinan solos antes de fijar un flujo.

Señales de que NO es el patron correcto:

- Necesitás que el paso B siempre ocurra después del paso A → usar Graph.
- Querés trazabilidad determinista → usar Graph.

```
function multiAgentPatterns() {
```

Graph

```
}
```

Cuando usarlo:

- El orden de los pasos importa y no cambia: $A \rightarrow B \rightarrow C$ siempre.
- Cada agente necesita la salida del anterior como input.
- Querés trazabilidad: saber exactamente qué agente hizo qué y en qué orden.

Señales de que es el patron correcto:

- Pipelines de procesamiento: extraer $\rightarrow$ validar $\rightarrow$ transformar $\rightarrow$ guardar.
- Flujos regulados donde el orden tiene consecuencias de negocio.
- "Primero buscar vuelos, después con esos datos consultar clima, después resumir."

Señales de que NO es el patron correcto:

- El orden puede variar según la tarea $\rightarrow$ usar Swarm.
- Los agentes viven en servicios distintos $\rightarrow$ combinar con A2A en los nodos.

```
function multiAgentPatterns() {
```

A2A

```
}
```

Cuando usarlo:

- El sub-agente corre en otro proceso, otro container o lo mantiene otro equipo.
- Querés que distintos agentes (incluso de distintos frameworks) colaboren via HTTP.
- Necesitás escalar cada agente de forma independiente.

Señales de que es el patron correcto:

- "El weather agent lo desplegó el equipo de data, yo solo lo consumo."
- Arquitecturas de microservicios donde cada agente es un servicio propio.
- Querés usar A2A como protocolo estándar (compatible con otros frameworks que implementen el spec de Google).

Señales de que NO es el patron correcto:

- Todo está en el mismo proceso → Agents as Tools es más simple y rápido.
- La latencia de red es un problema → evaluar si la separación vale el costo.

04

// CHAPTER

Agentcore



Amazon Bedrock AgentCore

Fully modular, secure and serverless agentic AI platform

Agents and tools

Any framework



Any model



All popular protocols



[AgentCore native resources](#)



Amazon Bedrock AgentCore

Fully modular, secure and serverless agentic AI platform

Agents and tools

Any framework



Any model



All popular protocols



[AgentCore native resources](#)

Build

Built-in tool: Browser

Interact with the web and internal domains using secure headless browsers.

Any model, framework, and popular browser automation libraries



Built-in tool: Code interpreter

Execute code securely across multiple languages in sandbox environments.

Common languages



Gateway

Connect agents to tools and data using API, Lambda and MCP targets.

Popular integrations



Identity

Manage agent access across AWS and 3P resources.

Any identity provider (IdP)



Memory

Maintain context across interactions.

Short-term Long-term Semantic memory strategy

User preference memory strategy Summary strategy

Policy

Establish cross-organization governance policies.

Policy engine Deterministic tool policies

Payments

Automate agent micro-transactions to pay for APIs and MCPs.

Supported identity provider (IdP)



Registry

Centralized way to discover and manage AI resources.

Agents MCP Servers Tools Skills Custom resources



Amazon Bedrock AgentCore

Fully modular, secure and serverless agentic AI platform

Agents and tools

Any framework



Any model



All popular protocols



[AgentCore native resources](#)

Build

Built-in tool: Browser

Interact with the web and internal domains using secure headless browsers.

Any model, framework, and popular browser automation libraries



Built-in tool: Code interpreter

Execute code securely across multiple languages in sandbox environments.

Common languages



Gateway

Connect agents to tools and data using API, Lambda and MCP targets.

Popular integrations



Identity

Manage agent access across AWS and 3P resources.

Any identity provider (IdP)



Memory

Maintain context across interactions.

Short-term Long-term Semantic memory strategy

User preference memory strategy Summary strategy

Policy

Establish cross-organization governance policies.

Policy engine Deterministic tool policies

Payments

Automate agent micro-transactions to pay for APIs and MCPs.

Supported identity provider (IdP)



Registry

Centralized way to discover and manage AI resources.

Agents MCP Servers Tools Skills Custom resources

Deploy

Runtime

Host agents and tools using any framework and any model on secure, serverless infrastructure.

Isolated sessions Serverless endpoints Embedded identity MCP & A2A Compatible Low cold start Rapid deployment Long-running async execution



Amazon Bedrock AgentCore

Fully modular, secure and serverless agentic AI platform

Agents and tools

Any framework



+ More

Any model



+ More

All popular protocols



+ More

AgentCore native resources

Build

Built-in tool: Browser

Interact with the web and internal domains using secure headless browsers.

Any model, framework, and popular browser automation libraries



+ More

Built-in tool: Code interpreter

Execute code securely across multiple languages in sandbox environments.

Common languages



Gateway

Connect agents to tools and data using API, Lambda and MCP targets.

Popular integrations



+ More

Identity

Manage agent access across AWS and 3P resources.

Any identity provider (IdP)



+ More

Memory

Maintain context across interactions.

Short-term Long-term Semantic memory strategy

User preference memory strategy Summary strategy

Policy

Establish cross-organization governance policies.

Policy engine Deterministic tool policies

Payments

Automate agent micro-transactions to pay for APIs and MCPs.

Supported identity provider (IdP)



Registry

Centralized way to discover and manage AI resources.

Agents MCP Servers Tools Skills Custom resources

Deploy

Runtime

Host agents and tools using any framework and any model on secure, serverless infrastructure.

Isolated sessions Serverless endpoints Embedded identity MCP & A2A Compatible Low cold start Rapid deployment Long-running async execution

Assess

Observability

Monitor and debug AI agents' performance in production environments.

Sessions Traces Spans Errors Latency

Evaluations

Evaluate your AI agents' performance.

Batch evaluations Online evaluation configurations

Custom evaluators

Refinements

Improve your agentic application

Recommendations A/B tests

// que hace cada uno

Strands vs AgentCore

[STRANDS]

Framework de agentes.

Define tools, modelos, hooks
Maneja el agent loop
Multi-provider de fabrica
Open source, corre donde quieras

[AGENTCORE]

Runtime de produccion en AWS.

Memory, Gateway, Identity, Browser
Observabilidad y deploy gestionado
Auth con IAM y Cognito
Strands corre adentro de AgentCore

// 4 que profundizamos

Los 3 componentes que vamos a usar

01

Runtime

Donde corre el agente. Empaqueta, escala y serve via HTTP.

02

Memory

Memoria persistente por usuario. Short-term y long-term.

03

Gateway

Expone APIs internas como tools sin filtrar credenciales al modelo.

```
// el mismo codigo, local y prod
```

AgentCore Runtime

```
~/travel-agent/agent.py
```



```
from bedrock_agentcore.runtime import BedrockAgentCoreApp

app = BedrockAgentCoreApp()

@app.entrypoint
def invoke(payload: dict) -> dict:
    response = agent(payload['prompt'])
    return {'result': str(response)}
```

```
function memoryTypes() {
```

Memoria corta vs larga.

```
}
```

```
// Short-term: contexto de la sesion actual, vive mientras dura la conversacion. Long-term:  
hechos sobre el usuario que persisten entre sesiones. Pregunta clave: si esto pasa entre dos  
sesiones distintas con un dia de distancia, lo necesito? Si si, va a long-term.
```

04

// CHAPTER

Gateway + Policies

Aislamos APIs internas detras de un Gateway corporativo. Aplicamos 5 politicas de viaje. Persistimos reservas en DynamoDB con use_aws.

```
function whyGateway() {
```

Por que un Gateway. ■

```
}
```

```
// El agente nunca toca APIs internas con credenciales corporativas. El Gateway intermedia: el
agente le pide funciones de negocio, el Gateway las traduce a llamadas autorizadas. Beneficio:
el modelo nunca ve un token de produccion. Cambios de auth, rate limit y auditoria viven en el
Gateway, no en el agente.
```

// gateway corporativo

Gateway **architecture** intermedia



// El agente le pide al Gateway funciones de negocio. El Gateway autentica, autoriza y traduce a llamadas internas.

```
// que te llevas del curso
```

De todo el **curso**

Strands: tools, agent loop, multi-provider. Funda todo lo demas.

MCP: tus tools son reusables fuera del agente que las creo.

AgentCore: memoria persistente y runtime de produccion en AWS.

Policies + Hooks: // el agente confiable para empresas reales.

Tu agente vive: Productivo.

\$./qa.sh --open

Questions?

// thanks for joining · let's talk

[WEB]

ricardoceci.dev

[SOCIAL]

@ricardoceci

[EMAIL]

ricardo@ricardoceci.dev